Given an array *taskMemory* of *n* positive integers representing memory required for each task, an array *taskType* of *n* positive integers representing task types, and an integer *maxMemory*, find the minimum time required to process all tasks.

Each task takes 1 unit of time. The server can process at most two tasks in parallel only if they are the same type and together require no more than *maxMemory* units.

Example

Suppose *n* = 4, *taskMemory* = [7, 2, 3, 9], *taskType* = [1, 2, 1, 3], and *maxMemory* = 10.

One efficient schedule is shown.

Task Pair	Task 1	Task 2	Task Type	Memory Requirement	Within Max Memory	Can Process in Parallel
1	0	2	Same	7 + 3 = 10	Yes	Yes
2	1	3	Different	9 + 2 = 11	No	No

Tasks 0 and 2 are processed concurrently, but the other two must be processed separately due to their memory requirements and because they are not the same type. The minimum amount of time required to process all the tasks is 3 units.

Function Description

Complete the function *getMinTime* in the editor with the following parameter(s):

int taskMemory[n]: the memory required by the tasks
int taskType[n]: the type of the tasks
int maxMemory: the maximum total memory that can be allocated to the tasks

Returns

int: the minimum time required to process all tasks

Constraints

- $1 \le n \le 2*10^5$

Constraints

- $1 \le n \le 2*10^5$
- $1 \le maxMemory \le 10^9$
- $1 \le taskMemory[i] \le maxMemory$
- $1 \le taskType[i] \le 10^9$

▶ Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN           FUNCTION
-----           --------
5         →     n = 5
1         →     taskMemory = [1, 2, 3, 4, 2]
2
3
4
2
5         →     n = 5
1         →     taskType = [1, 2, 1, 2, 3]
2
1
2
3
4         →     maxMemory = 4
```

Sample Output

```
4
```

Explanation

The first and the third tasks are processed in parallel. The other three tasks need to be processed individually. The second and fourth use too much memory together, and the fifth is a unique type.

```
1  > #include <bits/stdc++.h> ···
9
0    /*
1     * Complete the 'getMinTime' function below.
2     *
3     * The function is expected to return an INTEGER.
4     * The function accepts following parameters:
5     *  1. INTEGER_ARRAY taskMemory
6     *  2. INTEGER_ARRAY taskType
7     *  3. INTEGER maxMemory
8     */
9
0    int getMinTime(vector<int> taskMemory, vector<int> taskType, int maxMemory) {
1        int n=
2    }
3
4  > int main() ···
```

Given *n* processes that need to be executed. Among these *n* processes, *k* are classified as high-priority processes, with their indices (1-based) represented in the array as *high_priority[i]*.

An OS scheduler is responsible for overseeing the execution of all processes. When a scheduler assigns a set of processes to a processor, it has two options:

- If the assigned processes are greater than 1 and even, it can divide the array of processes, denoted as *p*, into two contiguous subarrays of equal length, *p1* and *p2*, such that *p = [p1, p2]*. The scheduler will then allocate *p1* to one processor and *p2* to another.
- Alternatively, the scheduler can choose to execute the assigned array of processes, *p*.

The time required for process execution is determined based on the following criteria:

1. If the assigned processes do not include any high-priority processes, the scheduler will take *normal_time* seconds to complete all the assigned processes.
2. If there are high-priority processes among the assigned tasks (denoted as x), it will take *(priority_time * x * l)* seconds to complete them, where *l* is the total number of assigned processes.

The total time required to execute all processes is the sum of the time taken by all processors for their assigned tasks. The task is to minimize the total execution time by optimizing the assignment of processes to processors within the operating system and return this minimum possible execution time.

Example
n = 4
k = 1
normal_time = 2
priority_time = 2
high_priority = [1]

One of the optimal ways is as follows:

- Assign a processor to [1, 2], which will execute the processes in (2 * 1 * 2) = 4 seconds.
- Assign a processor to [3, 4], which will execute the processes in 2 seconds.

Hence, the total time taken is 6 seconds.

k = 1
normal_time = 2
priority_time = 2
high_priority = [1]

One of the optimal ways is as follows:

- Assign a processor to [1, 2], which will execute the processes in (2 * 1 * 2) = 4 seconds.
- Assign a processor to [3, 4], which will execute the processes in 2 seconds.

Hence, the total time taken is 6 seconds.

Function Description

Complete the function *getMinimumTime* in the editor below.

getMinimumTime takes the following parameters:

int n: the total number of processes
int high_priority[k]: the indices of the high-priority processes
int normal_time: the time factor for completing non-high-priority processes
int priority_time: the time factor for completing high-priority processes

Returns

int: the minimum total execution time by optimizing the assignment of processes to processors within the operating system

Constraints

- $1 \leq n \leq 10^9$
- *n* is a power of 2
- $1 \leq k \leq min(n, 10^5)$
- $1 \leq high_priority[i] \leq n$
- $1 \leq normal_time, priority_time \leq 10^4$

int: the minimum total execution time by optimizing the assignment of processes to processors within the operating system

Constraints

- $1 \le n \le 10^9$
- n is a power of 2
- $1 \le k \le min(n, 10^5)$
- $1 \le high_priority[i] \le n$
- $1 \le normal_time, priority_time \le 10^4$

▶ **Input Format For Custom Testing**

▼ **Sample Case 0**

Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
4       →   n = 4
2       →   high_priority[] size k = 2
1       →   high_priority = [1, 4]
4
1       →   normal_time = 1
5       →   priority_time = 5
```

Sample Output

```
12
```

Explanation

One of the optimal approaches involves assigning each of the four processors to one of the four processes.

The time taken to complete a high-priority project will be (5 * 1 * 1) = 5 seconds.

As a result, the total time required to complete all the processes will be (5 + 1 + 1 + 5) = 12 seconds.

Language C++23

```
1  #include <bits/stdc++.h>
9  /*
10  * Complete the 'getMinimumTime' function below.
11  *
12  * The function is expected to return a LONG_INTEGER.
13  * The function accepts following parameters:
14  *  1. INTEGER n
15  *  2. INTEGER_ARRAY high_priority
16  *  3. INTEGER normal_time
17  *  4. INTEGER priority_time
18  */
19
20  long getMinimumTime(int n, vector<int> high_priority, int normal_time, int
    priority_time) {
21
22  }
23  int main()
```

Ln 9, Col 1 • Autocomplete Spaces: 4 Mode: Normal

Determine how many different ways you can allocate *m* images across *n* filters while keeping the total processing time at or below a maximum limit.

Each filter *i* takes *time[i]* seconds to process one image. When you assign *imagesAssigned[i]* images to filter *i*, that filter will take *(imagesAssigned[i] * time[i])* seconds to complete its work. The total processing time is the sum of processing times across all filters.

Your task is to count all possible configurations where:

- Each filter is assigned 0 or more images
- The total number of assigned images equals *m*
- The total processing time does not exceed *maxTime*

Since the result could be very large, return it modulo $(10^9 + 7)$.

Example
n = 3 filters
m = 3 images to process
maxTime = 3 seconds
time = [1, 1, 1] (each filter takes 1 second per image)

Valid configurations:

- [1, 1, 1]: Total time = 1×1 + 1×1 + 1×1 = 3 seconds
- [2, 1, 0]: Total time = 2×1 + 1×1 + 0×1 = 3 seconds
- [2, 0, 1]: Total time = 2×1 + 0×1 + 1×1 = 3 seconds
- [0, 2, 1]: Total time = 0×1 + 2×1 + 1×1 = 3 seconds
- [1, 2, 0]: Total time = 1×1 + 2×1 + 0×1 = 3 seconds
- [1, 0, 2]: Total time = 1×1 + 0×1 + 2×1 = 3 seconds
- [0, 1, 2]: Total time = 0×1 + 1×1 + 2×1 = 3 seconds
- [3, 0, 0]: Total time = 3×1 + 0×1 + 0×1 = 3 seconds
- [0, 3, 0]: Total time = 0×1 + 3×1 + 0×1 = 3 seconds
- [0, 0, 3]: Total time = 0×1 + 0×1 + 3×1 = 3 seconds

Function Description

Complete the function *getNumConfigurations* in the editor that takes the following parameter(s):

int n: the total number of image filters
int time[n]: the time taken by each image filter to filter one image
int m: the total number of images to be processed
int maxTime: the maximum processing time that can be spent on filtering all the images

Returns

int: the number of ways *m* images can be assigned to *n* image filters such that the total processing time does not exceed *maxTime*

Constraints

- $1 \le n, m \le 300$
- $0 \le maxTime, time[i] \le 300$

▶ Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN        FUNCTION
-----        --------
3       →    time[] size n = 3
0       →    time = [0, 1, 0]
1
0
3       →    m = 3
0       →    maxTime = 0
```

Sample Output

```
4
```

Explanation

The possible configurations for *imagesAssigned* are [3, 0, 0], [2, 0, 1], [1, 0, 2], and [0, 0, 3], where each configuration will take 0 seconds to filter the images.

▶ Sample Case 1

Language C++23

```
1  #include <bits/stdc++.h>
9
10 /*
11  * Complete the 'getNumConfigurations' function below.
12  *
13  * The function is expected to return an INTEGER.
14  * The function accepts following parameters:
15  *  1. INTEGER_ARRAY time
16  *  2. INTEGER m
17  *  3. INTEGER maxTime
18  */
19
20 int getNumConfigurations(vector<int> time, int m, int maxTime) {
21
22 }
23
24 int main()
```

Ln 9, Col 1 • Autocomplete Spaces: 4 Mode:

Test Results Run

Who normally manages object memory allocation and release in Python?

- ○ The Python runtime
- ◉ Only the programmer using free()
- ○ The operating system user
- ○ The text editor

Clear Selection

What is the difference between a class and an object?

- ○ An object is a blueprint, and a class is an instance created from it
- ○ They are two names for exactly the same thing in Python
- ○ A class stores only data, and an object stores only functions
- ◉ A class is a blueprint, and an object is an instance created from it

Clear Selection

What is the main purpose of a decorator in Python?

- (●) To modify or extend a function or class without changing its source code
- ( ) To delete a function after it runs
- ( ) To convert every function into a class
- ( ) To install a Python package

Clear Selection

What is the output of the following code?

```
s = {1, 2, 2, 3, 3, 3}
print(len(s))
```

- ○ 6, because a set keeps every value that was written inside the braces
- ○ 4
- ○ 1
- ◉ 3

Clear Selection

What is a cloud region?

- (●) A billing plan that sets the price for each resource
- ( ) A geographic area containing one or more data centres
- ( ) A user group
- ( ) A programming language

Clear Selection